AI-Assisted On-Call Queue: Budgeted Incident Detection for Cloud Microservices

Aishah Mohamed Alyammahi

Abstract

On-call engineers can read only a few alerts a day, and 82% of GAIA faults never write a log line. The system shows at most K items a day, in the order they become known, using memory jumps, slow leaks, and monitoring gaps. At K = 20 the jump detector catches 72.6% of 1,223 test faults, against 68.9% for a Prometheus rate rule and 0% for log severity. The jump detector wins all 8 walk-forward weeks against that rate rule, with fewer false items a day.

1. Motivation and objectives

A platform of many small services raises more alerts than one person can read. Real faults are missed because attention runs out, not because the metrics were silent. The users are the on-call engineer, who needs a short list that still contains the real faults, and the operator who turns Live on, who needs nothing paged by accident and a way to put a model back. The practical value is the daily budget. The queue shows at most K items, in the order a live system learns about them, so the team is not left choosing between a flood of pages and a pager that stays quiet.

Teams already cut repetition. Alertmanager groups alerts that share labels. PagerDuty and Opsgenie merge alerts that look alike or share an alias. None of those tools chooses which incidents fit into K readings a day, and none treats a rise in memory as an incident before a log alert has fired. The engineer reads the day's queue and marks an item Real or Noise. The operator installs the service, leaves shadow mode on until a pilot agrees, and is the only person who can turn Live on.

Each objective below is paired with the measurement this submission reports. The comparison that decides success is the budgeted jump queue against the rules a team already runs.

Objective	Target	Measured result
Beat the rate rule at K = 20	More faults caught than a Prometheus rate rule, with fewer false items	72.6% against 68.9%, and 1.0 against 2.1 false items a day. The jump detector (`jumps_tuned`) wins 8 of 8 walk-forward weeks against the Prometheus rate rule
Decide with no future data	No decision uses a later point, the end of a jump, or its peak	A jump is shown at the second confirming point. `tests/test_causality.py` checks that rule
Ship with shadow paging and a versioned model	Record what would have been paged until Live is armed. Models are versioned and can be rolled back	Shadow is the default. Live needs `APP_PASSWORD`, a channel, and the typed word LIVE, and only an ingest job may page. Models load as skops
Let another team run the tests	The checks run from the README on the committed results, with no paid API	530 passed, 22 skipped; line coverage 86.93%; `run_validate.py` matches the printed figures

2. Architecture and implementation

2.1 Architecture

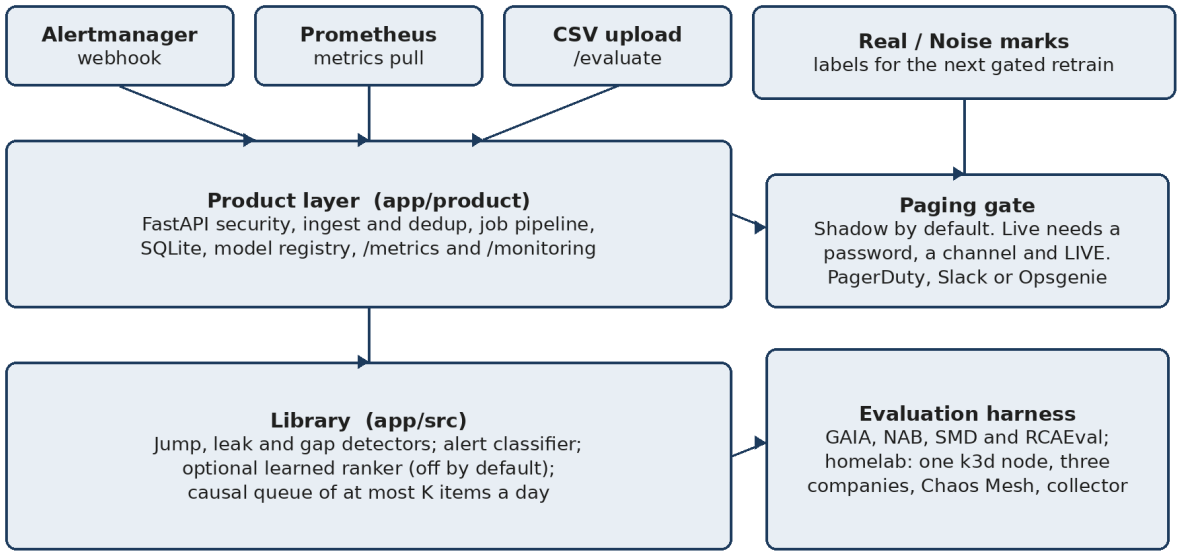


Figure 1. Alertmanager and Prometheus feed app/product. Detection is in app/src. The queue is stored in SQLite. Pages leave only from ingest jobs

Alertmanager posts firing alerts into `app/product`. Prometheus supplies the memory series, and the same series can be uploaded as CSV. The product is a FastAPI service. It checks the caller, drops duplicate webhook rows, runs one scoring job, and writes the queue to SQLite. Detection lives in `app/src`: the jump rule, the slow-leak rule, the monitoring-gap rule, and the daily budget. A page is sent only from an ingest job, and only after Live is armed. The HTTP routes are listed in `docs/report_appendices_full.md`.

The stack is Python, FastAPI, scikit-learn, pandas, SQLite, Docker and GitHub Actions. Dependencies are locked with uv. Each job stores the queue it built and the model version that scored the alerts. A Real or Noise mark is kept as a label. It does not itself send a page.

2.2 How the queue decides

The queue is first come, first served, and the day stops at K. A memory reading more than 0.5 GB above the previous 15-minute median is hot. The jump is shown at the second hot point within two minutes, and only when the rise known at that moment meets the threshold learned on the first week (0.62 GB on GAIA). A slow leak uses a three-hour median. A monitoring gap is ten minutes with no point. Log alerts may use at most a quarter of K. An alert that arrives while a jump on the same service is still open rides on that jump and takes no extra slot. No decision uses a later reading, the time the jump eventually ends, or the height it eventually reaches. A saved model is loaded as skops. A pickle file is refused even when its name still ends in `.pkl`. Shadow stays on until the operator sets `APP_PASSWORD`, names a channel, and types the word LIVE.

2.3 Change from the proposal

The significant change from the proposal is that the submitted system is the **budgeted jump queue**. An early hope of catching 95% of faults does not fit this data. GAIA injects about 22 faults a day, so 84.9% is the best that K = 20 can reach, and 82% of the faults never write a log line. The log classifier is trained and gated. On GAIA it is not what catches the faults. An LLM summary of an incident stayed out of scope, as the proposal said.

3. Testing, evaluation and results

Training uses the first 7 days only. The test period is the next 55 days and holds 1,223 faults. Days are not shuffled. A fault label scores a queue that has already been built. It is not an input to the decision. A fault counts as caught when a shown jump or leak on its service overlaps the injection, or a shown alert carries its id. A monitoring-gap item does not count as a catch.

Table 1. Method at K = 20 on the GAIA test period (1,223 faults). Counts are in `app/results/queue_eval_gaia.csv`.

Method at K = 20	Faults caught	False items a day
Log severity	0%	—
Prometheus rate rule	68.9%	2.1
Jump detector	72.6%	1.0
Best possible on this data	84.9%	—

The jump detector is the row `jumps_tuned`: 888 of 1,223 faults (72.6%) and 1.0 false item a day. The Prometheus rate rule is `delta_5m`: 843 of 1,223 (68.9%) and 2.1 false items a day. Log severity catches 0 of 1,223. The jump detector (`jumps_tuned`) wins all 8 walk-forward weeks against the Prometheus rate rule (`delta_5m`), and on the 55-day test it shows 1.0 false item a day against the rate rule's 2.1. Walk-forward learns every threshold again from the past only, then scores the next week once. Appendix A has that detector and, separately, the full queue on the same weeks. The full queue is a different comparison, and it does not win every week. Median time from the start of a fault to the confirming point is 0.72 minutes.

About 22 faults are injected a day, more than 20 slots on 34 of the 55 test days, which is why the best possible result at this budget is 84.9% rather than 100%. When the full queue misses, Appendix A shows that the misses are mostly a monitoring gap or a day whose 20 slots were already full. The jump rule itself fails on only a small share of those misses.

On this commit, `uv run pytest -q --cov --cov-report=term --cov-fail-under=85` in `app/` reported 530 passed and 22 skipped. Line coverage is 86.93%. Twelve skips are the Helm chart tests, which run only when `helm` is installed. A load test of 50 concurrent users recorded p95 0.32 s and no errors (`docs/report_appendices_full.md`). `uv run python run_validate.py` matched the printed figures (300 checked numbers).

The classifier on GAIA catches fewer faults than the same queue with no log alerts, so the gate leaves it inactive. Results on RCAEval, NAB and SMD are in `docs/report_appendices_full.md`.

4. Deliverables, setup and use

The repository is https://github.com/amamsy90/alert-fatigue-reduction. Submitted commit: `0ca024580222b0445544fc34931b2c68cc9bc7f9`. The delivery is the application, the tests, and this report. From a checkout, run the four commands below. Exact steps are in the README. Shadow mode is the default, so a job records what it would have paged, and an evaluation does not page. The tests and `run_validate.py` read the committed files in `app/results/`. GAIA itself is not in git. `tools/fetch_gaia.py` downloads it (about 1.8 GB) and checks each file by SHA-256. No paid API is required. The full appendices are in `docs/report_appendices_full.md`. PagerDuty, Slack and Opsgenie stay unused while shadow mode is on.

```
cd app
uv sync --group dev
uv run pytest -q
uv run python run_validate.py
```

5. Conclusion

The budgeted jump queue beats the rules a team already runs, at K items a day, without later data. At K = 20 the jump detector catches 72.6% of 1,223 GAIA test faults, against 68.9% for a Prometheus rate rule and 0% for log severity, with 1.0 false item a day against the rate rule's 2.1. That detector wins all 8 walk-forward weeks against the rate rule. Log severity catches none of these faults, because most of them never log.

GAIA's headline is one fault type, a memory injection. The margin over the rate rule is modest: 3.7 points for the jump detector. The full queue does not win every week (4 wins, 3 losses and 1 tie; Appendix A). The log classifier adds no catches on GAIA, so the gate leaves it inactive. Leave-one-company-out for the homelab ranker is a DROP (Appendix B). About 22 faults a day sets an 84.9% ceiling at K = 20.

Next is a shadow-mode pilot on a real team's alerts. SMD and RCAEval scores for other fault types are in `docs/report_appendices_full.md`.

Appendix A. GAIA headline evidence

The K = 20 comparison, the misses of the full queue, and the walk-forward weeks cited in §3 and §5. The full write-up is in `docs/report_appendices_full.md`.

System (K = 20 items a day)	Caught	Share of best possible	Items / day	False / day	Precision
Log severity + exact duplicates	0.0%	0%	0.8	0.8	0%
Static rule: memory > week-1 p99	57.2%	67%	16.0	3.0	81%
Rate rule: `delta(mem[5m]) > 0.5 GB`	68.9%	81%	17.4	2.1	88%
Jump rule, first come	67.8%	80%	17.5	2.4	86%
Jump rule + learned threshold (detector alone)	**72.6%**	**86%**	17.1	**1.0**	**94%**
Full queue with the classifier (jumps + leaks + gaps + classifier alerts; refused by the gate on GAIA)	**69.6%**	**82%**	17.7	1.8	90%
Shipped configuration (the gate keeps the classifier out: jumps + leaks + gaps + log alerts by severity)	**71.0%**	**84%**	17.6	1.4	92%
Old headline: day's jumps ranked by final peak (not real time)	*72.9%*	*86%*	*17.5*	*1.3*	*93%*

The full queue misses 372 of the 1,223 test faults at K = 20 (`results/miss_causes_gaia.csv`):

Cause	Faults	Share of misses
No metric data during the fault (monitoring outage)	166	45%
A qualifying jump existed, but the day's 20 slots were used	198	53%
Metric data, but no jump overlapped	4	1%
A jump overlapped, but below the confirmation threshold	4	1%

Walk-forward at K = 20. The detector beats the rate rule in 8 of 8 weeks. The full queue has 4 wins, 3 losses and 1 tie.

Fold	Test week	Faults	With metric data	Rate rule	Detector alone	Full queue
1	8–15 Jul	161	100%	77.6%	84.5%	82.0%
2	15–22 Jul	134	91%	73.9%	81.3%	74.6%
3	22–29 Jul	162	94%	72.2%	77.2%	71.0%
4	29 Jul–5 Aug	163	82%	67.5%	69.3%	67.5%
5	5–12 Aug	162	84%	69.1%	71.6%	71.0%
6	12–19 Aug	139	92%	75.5%	79.1%	74.1%
7	19–26 Aug	173	**61%**	51.4%	52.0%	50.9%
8	26–31 Aug	129	91%	78.3%	83.7%	79.1%

Fold	Test week	Faults	With metric data	Rate rule	Detector alone	Full queue
Mean				**70.7%**	**74.8%**	**71.3%** (95% CI 64.5–76.7%)

Appendix B. Home-lab leave-one-company-out

K = 20. Train on the other two companies and test on the held-out company (`ranker/leave_one_company_out.json`). Each comparison is a DROP.

Company	Baseline	Ranker	CI	Keep
acme	32 of 220	32	[0.0, 0.0]	DROP
borealis	25 of 159	25	[0.0, 0.0]	DROP
cassini	21 of 180	21	[0.0, 0.0]	DROP